CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 6 & 7 B-Trees

Midterm summary · CLRS Chapter 18

Muharib

What's inside

01 What is a B-tree?

02 Bounds: how many keys per node?

03 Operations and their cost

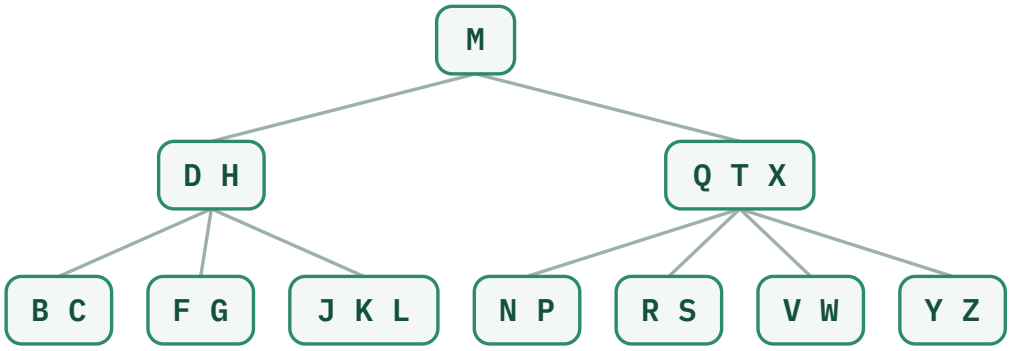
04 Insertion

05 Slide exercise: t = 3

06 Deletion

01 What is a B-tree?

- A **balanced** search tree; each node holds several keys and can have more than two children.
- **All leaves are at the same depth** (the tree's height h).
- A node with `n` keys has `n + 1` children.
- Keys inside a node are in non-decreasing order.
- Designed for disks (magnetic disks / secondary storage). Also called a balanced sort tree.



B-tree	Binary tree
Storage on disk	Storage in RAM
More than two children	At most two children
Always balanced	Not necessarily

02 Bounds: how many keys per node?

The slides use the **minimum degree** `t ≥ 2`:

	Minimum	Maximum
Keys, non-root node	`t − 1`	`2t − 1`
Children, non-root internal node	`t`	`2t`
Root	1 key	`2t − 1` keys

```
t = 2:  1 to 3 keys,  2 to 4 children   (2-3-4 tree)
```

```
t = 3:  2 to 5 keys,  3 to 6 children
```

A node with `2t − 1` keys is **full**.

> **Mind the naming in the midterm**
>
> Midterms write "B-tree of degree 3" or "of order 5" meaning the **maximum number of children** `m`, so at most `m − 1` keys: degree 3 = at most 2 keys (a 2-3 tree), order 5 = at most 4 keys. This is not the slides' `t`.

03 Operations and their cost

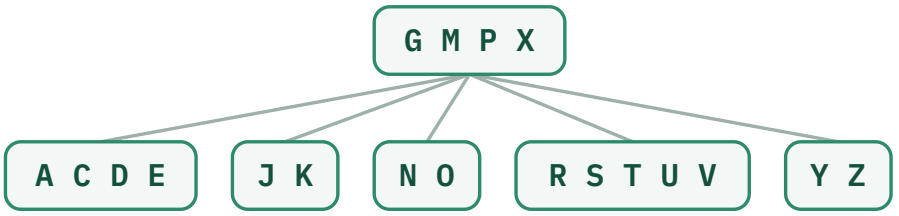
Operation	Idea	Cost
B-TREE-SEARCH(x, k)	Scan the node until `k ≤ key`; if found return it, otherwise go down to the right child	`O(t)` per node · `O(log_t n)` nodes
B-TREE-CREATE(T)	An empty leaf node becomes the root	`O(1)` disk and `O(1)` CPU
B-TREE-INSERT	Always inserts into a leaf	`O(t log_t n)`
B-TREE-DELETE	From any node, keeping the minimum size	`O(t log_t n)`

04 Insertion

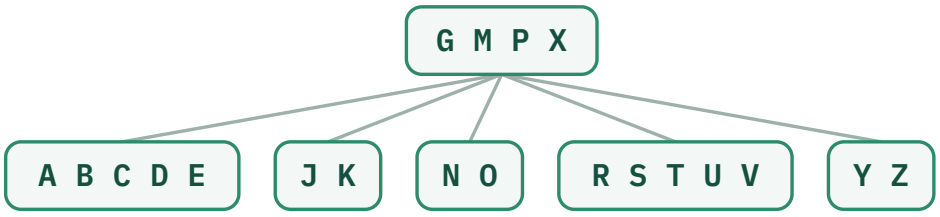
- We do not create a new leaf; the key goes into an existing leaf.
- If a node is full (`2t − 1`): **split** it around its median: the median moves up to the parent and the rest becomes two nodes of `t − 1` keys each.
- If the parent is full too, split it as well, up to the root. Splitting the root increases the height by one (the tree grows at the top).
- The slides' (CLRS) way: split every full node met **on the way down**, before reaching the leaf (single pass).

Slide example (t = 3): insert B, Q, L, F

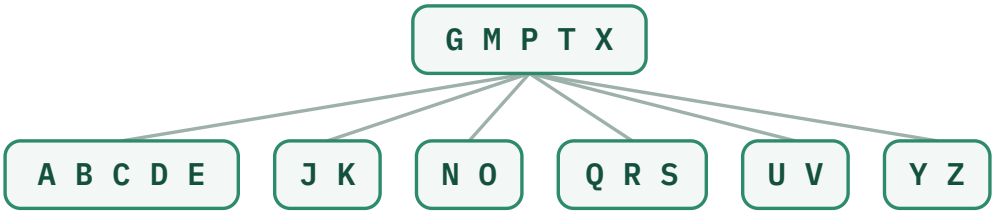
Initial tree:



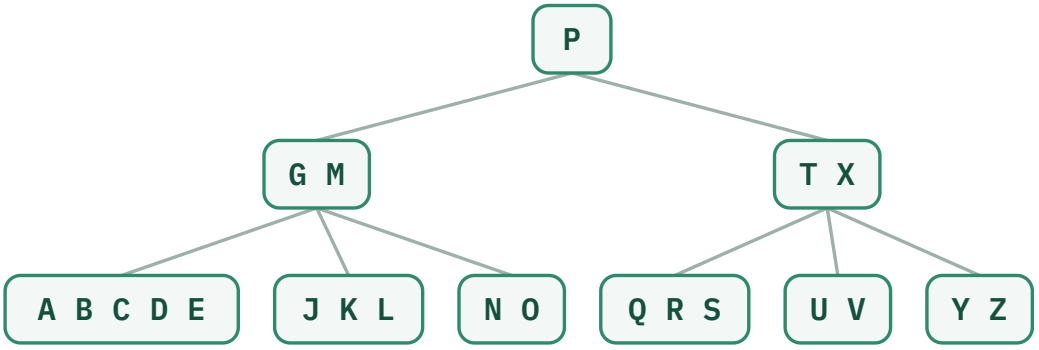
(b) B: leaf `A C D E` is not full, so B goes straight in.



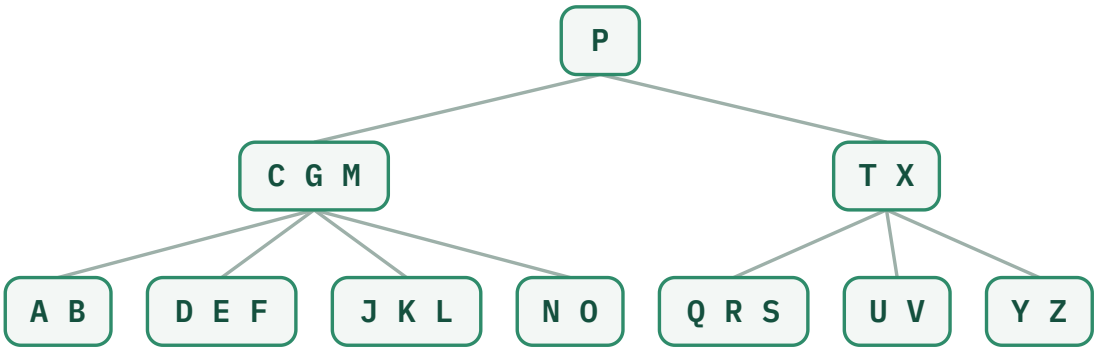
(c) Q: leaf `R S T U V` is full → split: `T` moves up to the root, then Q goes into `Q R S`.



(d) L: the root `G M P T X` is full → split the root: `P` becomes the new root and the height grows. Then L goes into `J K L`.



(e) F: leaf `A B C D E` is full → split: `C` moves up and F goes into `D E F`.



05 Slide exercise: t = 3

```
Insert: 3, 7, 9, 23, 45, 1, 5, 14, 25,
```

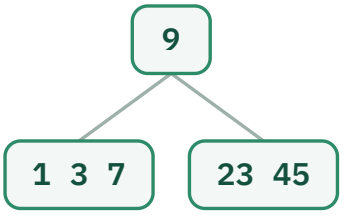
```
        24, 13, 11, 8, 19, 4, 31, 35, 56
```

```
t = 3  →  max 5 keys per node,  median = 3rd key
```

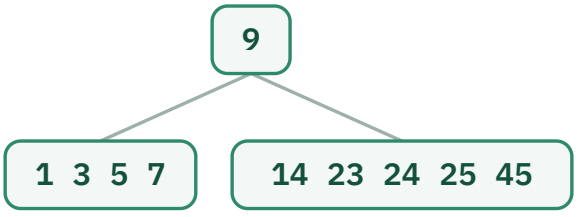
3, 7, 9, 23, 45 go into one leaf, which becomes full:



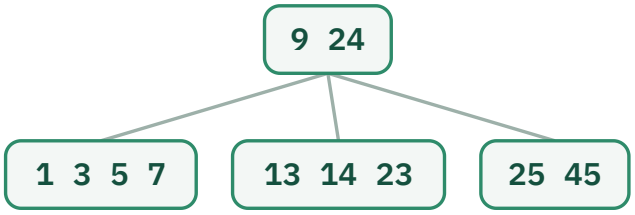
1: the root is full → split around 9:



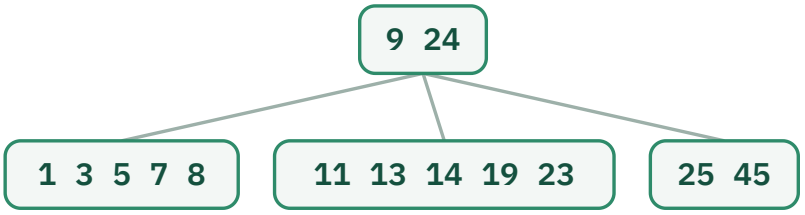
5, 14, 25, 24 go in without a split:



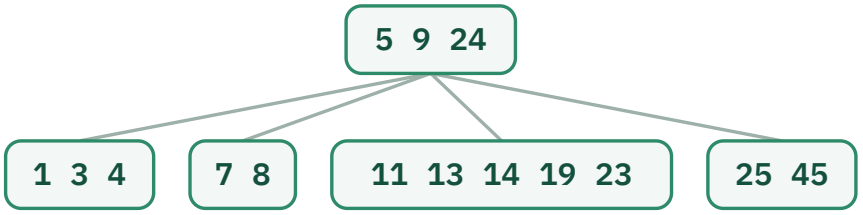
13: leaf `14 23 24 25 45` is full → 24 moves up:



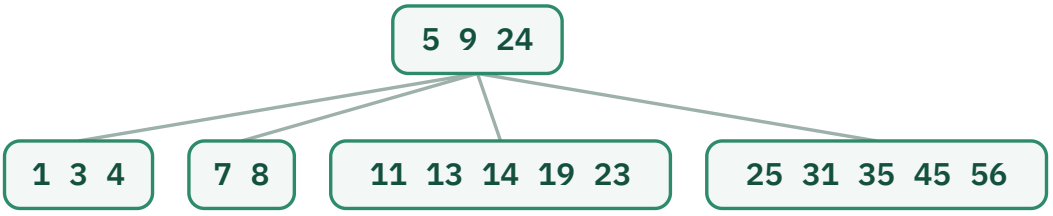
11, 8, 19 without a split:



4: leaf `1 3 5 7 8` is full → 5 moves up:



31, 35, 56 without a split. Final tree:



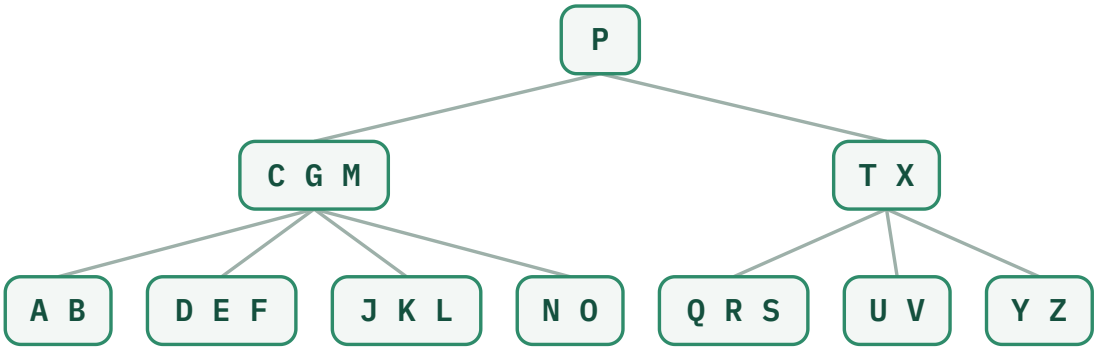
06 Deletion

We can delete from any node, but no node (except the root) may drop below `t - 1` keys.

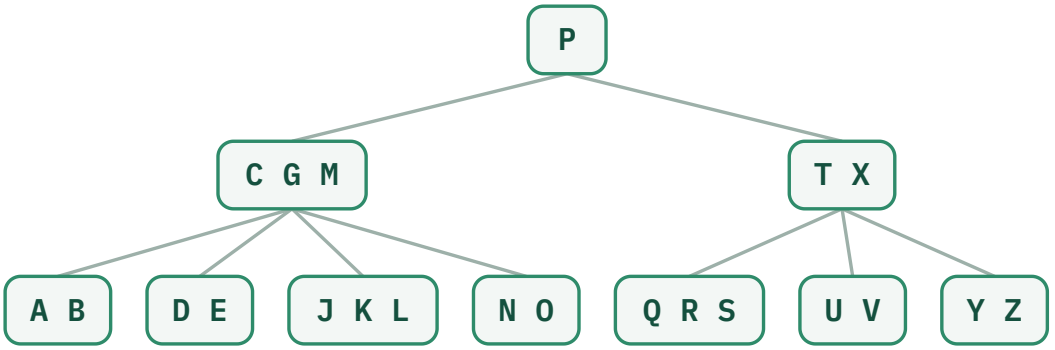
The fix is to merge nodes or borrow from a sibling.

Case	When	What to do
1	Key in a leaf with `≥ t` keys	Just delete it
2a	Key in an internal node; the child before it has `≥ t`	Replace it with its **predecessor** (largest key in the left child) and delete that below
2b	The child after it has `≥ t`	Replace it with its **successor** (smallest key in the right child)
2c	Both children have `t − 1`	**Merge**: left child + key + right child into one node, then delete the key from it
3	Key not in this node, and the child we descend into has only `t − 1`	Grow it before descending: **3a**: a sibling has `≥ t` → borrow: a key from the parent comes down and a key from the sibling goes up **3b**: all siblings have `t − 1` → merge with a sibling and a key from the parent

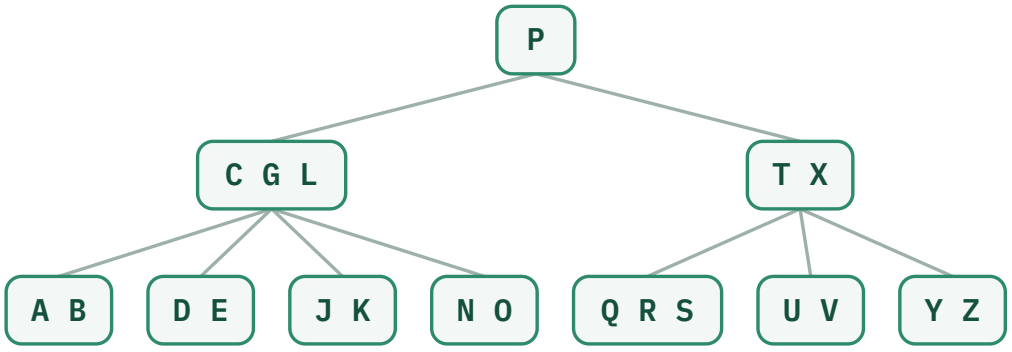
Slide example (t = 3)



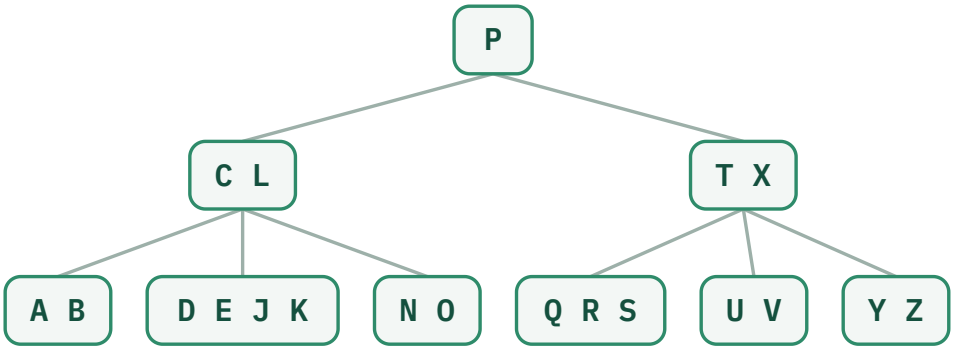
Case 1, delete F: leaf `D E F` has 3 ≥ t:



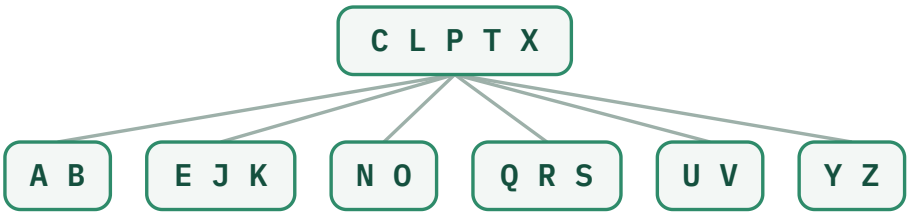
Case 2a, delete M: the left child `J K L` has 3 → L replaces M:



Case 2c, delete G: `D E` and `J K` have 2 each → merge into `D E G J K` and delete G:



Case 3b, delete D: descending from `P` to child `C L`, which has only 2, and its sibling `T X` also has 2 → merge both with P; the old root disappears (the height shrinks). Then D is deleted from the leaf:



The slide text for deleting D repeats the description of case 2c (combining DE and JK and pushing down G). The figure here follows the steps of case 3b.

Case 3a, delete B: leaf `A B` has 2 and its sibling `E J K` has 3 → C comes down and E goes up in its place:

